Task 5: Writing Join Queries, Equivalent, AND/OR Recursive Queries

Date: 9/9/25

Aim:- To implement and execute Join queries, equivalent queries, and recursive queries.

TYPES of Joins in SQL:

1. Inner Join:- Returns records that have matching values in both tables.

Syntax: Select column_name) from table1 INNER JOIN table2 on table1. column_name = table2. column_name;

2. Left Outer Join:- Returns all records from the left table, and the matched records from the right table.

Syntax: Select column_name(s) from table1 LEFT JOIN table2 ON table1. column_name = table2. column_name;

3. Right outer Join: Return all records from the right table, and the matched records from the left table.

Syntax: select column_name(s) from table1 RIGHT JOIN table2 ON table1. column_name = table2. column_name

4. Full Outer Join: Returns all records when there is a match in either left or right table.

Syntax: Select column_name(s) from table1 full outer Join table2 ON table1. column_name = table2. column_name;

1. JOIN QUERIES

Create Tables

```
create table customer(
    customerID int primary key,
    name varchar(50),
    address varchar(100)
);
create table bank_account (
    account_number int primary key;
    customerID int,
    balance int,
    category varchar(50),
    foreign key(customerID) references customer(customerID)
);
create table branch(
    branchID int primary key,
    branchName varchar(50),
);
```

2. Insert Sample data

```
insert into customer(customerID, name, address) values
(101, 'Ram kumar', 'chennai');
insert into customer (customerID, name, address) values
(102, 'vijay Rao', 'hyderabad');
insert into customer (customerID, name, address) values
(103, 'vasu Reddy', 'vizag');
insert into customer (customerID, name, address) values
(104, 'Vinay kumar', 'chennai');
insert into customer (customerID, name, address)
values (105, 'Rohit', 'Delhi');
insert into customer (customerID, name, address)
values (106, 'Varun', 'Kerla');
```

```
insert into bank-account (account_number, customerID, balance,
category) values (1001, 101, 15000, 'savings');
insert into bank-account (account_number, customerID,
balance, category) values (1002, 102, 0, 'current');
insert into bank-account (account_number, customerID,
balance, category) values (1003, 103, 5000, 'savings');
insert into bank-account (account_number, customerID,
balance, category) values (1004, 105, 2000, 'current');

insert into branch (branchID, branchName) values
(1, 'Chennai Branch');
insert into branch (branchID, branchName) values
(2, 'Hyderabad Branch');
insert into branch (branchID, branchName) values
(3, 'Vizag Branch');
```

3. Join Queries:

(a) Inner Join :-

Query :-
```
Select c.name, b.account_number from
customer c inner join bank_account b ON
c.customerID = b.customerID;
```

Output:

Name	account_number
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004

(b) Left Join

Query:- Select c.name, b.account_number from customer c left join bank_account b ON c.customerID = b.customerID;

output:-

name	account_number
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004
Rohit Sharma	NULL

(c) Right Join:-

Query:- select c.name, b.account_number from customer c Right Join bank_account b ON c.customerID = b.customerID;

output:-

name	account_number
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004

(d) Full outer Join:-

Query:- Select c.name, b.account_number from customer c Full outer join bank_account b ON c.customerID = b.customerID;

name	account_number
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004
Rohit Sharma	NULL

Equivalent Query

(a) using Join

Query:- Select C.name AS customerName, b.account_number AS Account number from customer c Join bank_account b on c.customerID = b.customer ID;

Output:-

customerName	AccountNumber
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004

(b) using Sub Query

Query:- Select C.name AS customerName,(Select b.account_number from bank_account b where b.customerID = c.customer ID Limit 1)AS Account Number from customer C;

Output:-

CostomerName	AccountNumber
Ram Kumar	1001
Vijay Rao	1002
Vasu Reddy	1003
Vinay Kumar	1004
Rohit sharma	NULL

5. Recursive Query:-

Query:- With Recursion Referal Hierarchy AS (Select customerID, referred By ID from customer where referred By ID is NOT NULL UNION.

select C. CustomerID, C. referred By ID from customer C Join Referal Hierarchy rh on C. referred By ID = rh. customer ID

select * from Referral Hierarchy;

output:-

customerID	referredByID
102	101
103	102
104	103

VEL TECH	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	14
SIGN WITH DATE	

Result:- The implementation of SQL commands using Joins and Recursive Queries are executed successfully.